Capstone Project Proposal

An Adaptive Defense Framework Against GenAI-Driven Web Payload Polymorphism Using Model Context Protocol (MCP)

Student Name: Nic Leighty

Semester: Fall 2026

Advisor Name: Jiazhen Zhou

Individual/Group: Individual

Total Credits: 6

Client: None

Abstract

The democratization of generative artificial intelligence (GenAI) has significantly lowered the barrier for automated exploit generation, enabling adversaries to dynamically mutate web payloads—such as SQL injection (SQLi) and cross-site scripting (XSS)—to systematically evade traditional, signature-based web application firewalls (WAFs). Legacy manual rule mitigation cycles are fundamentally ill-equipped to counter the scale and speed of this automated polymorphism. To bridge this defensive gap, this capstone project introduces an autonomous, self-healing application defense framework orchestrated via the open-standard Model Context Protocol (MCP). Operating entirely within a zero-cost, localized containerized architecture, the framework establishes a continuous closed-loop testing environment consisting of an open-source reverse proxy WAF (Nginx/ModSecurity) and a deliberately vulnerable target application (OWASP Juice Shop). An offensive mutation engine uses a local large language model (LLM) via Ollama to generate and launch sequential waves of polymorphic web exploits. To automate remediation, a custom Python-based MCP server exposes localized firewall tools to a blue-team AI agent via standardized JSON-RPC protocols. Upon crossing an endpoint breach threshold, the defensive agent ingests the filtered log data, identifies the semantic mutation pattern, and injects an optimized, idempotent security rule to permanently patch the vulnerability vector. Rather than

relying on static before-and-after metrics, the system's efficacy is continuously measured via a dedicated external test harness evaluating mean time to mitigation (MTTM), the exponential bypass decay rate (α), the rule generality index (RGI), and the regressive false positive rate (RFPR) against legitimate user traffic. This research demonstrates a scalable, LLM-agnostic paradigm for real-time security engineering capable of hardening application-layer defenses against adaptive, AI-driven threats.

Introduction and Problem Statement

Modern web application firewalls (WAFs) rely heavily on signature-based detection patterns (such as regular expressions) to identify and block malicious web traffic, including SQL injection, cross-site scripting (XSS), and remote code execution. However, the democratization of large language models (LLMs) has given adversaries the ability to generate highly sophisticated, polymorphic mutations of these payloads at scale. By dynamically altering encoding, tokenization, and syntax, generative AI can create exploits that bypass traditional static WAF rules while preserving the underlying exploit execution.

Current manual intervention cycles for updating WAF rules are too slow to counter automated AI-driven mutation attacks. There is an urgent need for an automated, self-healing defensive architecture that can detect novel, obfuscated payloads and dynamically update web application defenses in real-time.

Project Objectives

This capstone project aims to design, implement, and evaluate an automated, closed-loop security system that leverages the Model Context Protocol (MCP) to autonomously protect web applications from GenAI-mutated exploits.

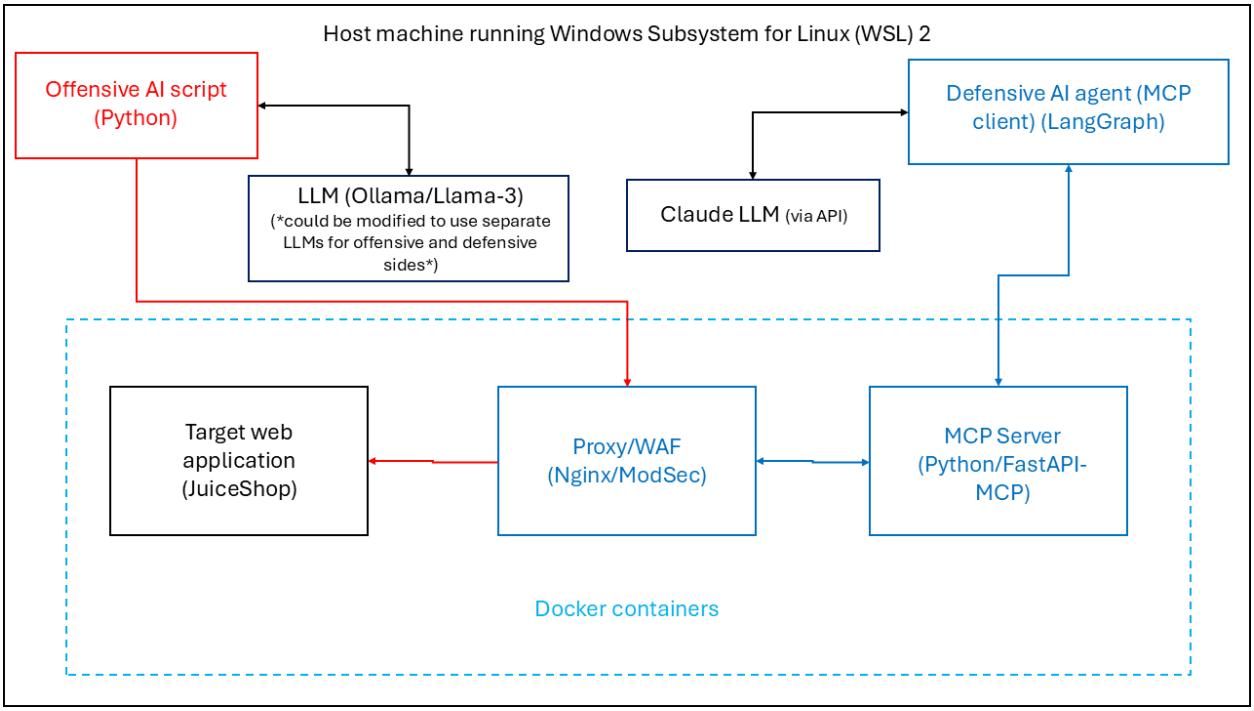
The specific engineering goals are:

1. **Develop a polymorphic fuzzer:** Build a local pipeline that uses an open-source LLM to generate heavily obfuscated variations of classic web application exploits.
2. **Build an MCP defensive server:** Implement a custom Model Context Protocol server that exposes WAF telemetry (logs) and WAF configuration controls as standardized tools for an AI agent.
3. **Implement a self-healing defensive agent:** Deploy an autonomous blue-team agent that analyzes successful WAF bypasses and dynamically generates new filtering rules to neutralize the mutation vectors.
4. **Evaluate system efficacy:** Benchmark the framework's performance using mean time to mitigation, bypass decay rate, rule generality index, and regressive false positive rate (explained in "Evaluation and Success Metrics").

Proposed Architecture & Methodology

The entire framework will be built using a containerized local microservices architecture to ensure ~~zero financial cost for the purposes of this project and~~ absolute LLM-agnostic flexibility.

Updated diagram using Claude for the defensive side LLM (20261004):



Component Breakdown

- The target: A local instance of a deliberately vulnerable web application (e.g., OWASP Juice Shop) hosted inside a Docker container.
- The firewall: An open-source reverse proxy WAF (Nginx with ModSecurity utilizing the OWASP Core Rule Set) placed in front of the target app.
- The adversary (simulation): A Python execution script that feeds standard payloads to a local instance of an LLM (Llama-3 via the Ollama wrapper), tasks it with obfuscating them, and fires them at the application.
- The defense standard (MCP): A custom Python-based (using FastMCP from FastAPI) MCP server that acts as a bridge. It converts specific tasks into standardized JSON-RPC tools that the LangGraph-powered defensive AI agent can call, specifically:
    - `read_waf_logs()`
    - `test_waf_configuration()`

- `write_idempotent_rule()`
- `reload_waf()`

~~Note that in the current proposed configuration, the intent is to use the same local LLM (Ollama/Llama-3) for both the offensive and defensive sides. However, via MCP, this could easily be expanded such that the defensive side uses a different LLM (such as Claude), while the offensive script remains attached to the archetypal local LLM.~~ The defensive side now uses Claude, while the offensive script remains attached to the archetypal local LLM. Using MCP on the defensive side promises flexibility in terms of LLM selection for the defensive side; the defensive LLM could easily be changed in future implementations of this project. Due to limitations in cloud-based LLMs being used for cyber offense (cloud-enforced guardrails), the scope of this project aims for the offensive LLM script to simply connect directly to a statically assigned LLM and not have a modular LLM connection that would be entailed with the use of MCP.

Note also that the intent is for the system to use threshold triggering by targeted endpoint to trigger the defensive side to act. A minimum viable product, however, would simply entail using a global count threshold of breaches that is endpoint independent.

Evaluation and Success Metrics

The system is also intended to have a test harness to enable gathering metrics to test the system's efficacy. This test harness shall be implemented such as to avoid compromising the core security code but rather operate at an independent "layer."

Consistent with the loop operation of the system, to validate its academic and practical utility, it will be evaluated as a dynamic, continuous system across sequential "attack waves" ($W_1, W_2, \ldots, W_n$). An attack wave is defined as a fixed batch of 50 unique polymorphic payloads generated by the offensive LLM. The system's efficacy will be gauged using four continuous telemetry vectors:

1. Mean Time to Mitigation (MTTM):

$$MTTM = T_{\text{deploy}} - T_{\text{threshold}}$$

This measures the agility and speed of the closed-loop defense mechanism. It calculates the temporal duration between the threshold trigger being tripped by a novel mutation vector and the successful automated compilation, validation, and deployment of the corrective WAF rule. Hence, T_{deploy} represents the time of deployment of the corrective WAF rule, while $T_{\text{threshold}}$ represents the time when the corresponding threshold trigger was tripped. Decreasing $MTTM$ across successive generations as the agent context window optimizes would illustrate success.

2. Bypass Decay Rate (α):

$$\alpha = \frac{B(W_n) - B(W_{n\text{-}1})}{B(W_{n\text{-}1})}$$

Let $B(W_n)$ be the number of successful bypasses in wave n. The bypass decay rate measures the system's trajectory toward full convergence (zero bypasses). The primary metric for defensive adaptation is the decay of successful exploit bypasses over sequential attack waves. If the defensive agent is truly identifying underlying semantic patterns (rather than just playing whack-a-mole with specific characters), the bypass rate should plummet drastically from wave to wave. A steep, negative slope ($\alpha < 0$) would indicate that the WAF is successfully actively hardening against future unseen mutations of attack vectors.

3. Rule Generality Index (RGI):

$$\text{RGI} = \frac{V_{\text{UBL}}}{V_{\text{TU}}} * 100$$

$V_{\text{UBL}} = \text{unseen variants blocked by legacy custom rules}$

$V_{\text{TU}} = \text{total unseen variants introduced}$

A major risk of using GenAI for defense is that it might write a rule that is too specific (e.g., blocking the exact spelling of one payload variant). To prove the agent is writing generalized behavioral filters, the Rule Generality Index tracks how many *new* mutations in Wave $n + 1$ are successfully blocked by a rule that was generated back in Wave n without the agent needing to intervene again.

4. Regressive False Positive Rate (RFPR):

$$\text{RFPR} = \frac{R_{\text{BFB}}}{R_{\text{TB}}} * 100$$

R_{BFB} = number of benign requests falesly blocked

R_{TB} = total number of benign requests sent

The goal is for RFPR to be 0% across all automated tuning cycles (every WAF reload). A self-healing firewall is useless if its automated patches break the production website. To measure system safety, a static control group of 100 benign, complex web requests (standard user traffic, nested JSON, heavy cookies) will be fired at the web app immediately following every single automated WAF reload.

Note that these metrics are subject to change.

Timeline

Sprint Dates	Goals
Sept. 2 – Sept. 15	**Environment Setup** • Deploy Docker containers for Nginx/ModSecurity and OWASP Juice Shop • Set up base logging and configure an AI generated rules configuration target (`ai_generated_rules.conf`)
Sept. 16 – Sept. 29	**Attacker Pipeline and Rest Harness** • Set up local Ollama instance for payload mutation • Script the test orchestrator to manage request batches and base metrics
Sept. 30 – Oct. 13	**MCP Server & Defenses** • Build Python MCP server to expose logs and rule-writing capabilities • Implement endpoint parsing and log-threshold tracking logic
Oct. 14 – Oct. 27	**Agent Integration and Dry Runs**

	• Connect defensive AI agent (LangGraph) to MCP tools • Enforce structured JSON outputs with static rule IDs for idempotency
Oct. 28 – Nov. 10	**Automated Testing and Drafting Report** • Run multi-wave attack loops to collect metrics ($MTTM, \alpha, RGI, RFPR$) • Compile and finalize the first rough draft of the report
Nov. 11 – Nov. 24	**Report Finalization and Submission** • Produce final data charts • Polish document and submit the final report to the graduate coordinator
Nov. 25 – Dec. 8	**Presentation Prep & Rehearsal** • Create presentation slides and record an architectural demo clip • Practice the presentation
Capstone Presentation Day (Date: TBD)	Present the capstone presentation

Note that, in the theme of agile development, this timeline is subject to adjustments based on operational changes to complete the project.

Resource Requirements

- Hardware: Standard development laptop (16GB+ RAM recommended for running local LLMs smoothly via Ollama)
- Software/frameworks used: Docker, Python 3.11+ (to include FastAPI-MCP), Ollama (Llama 3), LangGraph, Nginx ModSecurity
    - Github used for version control/project management
    - OneNote used for internal documentation/note management
- Financial budget: $0.00 (using open-source software and local compute)

- Implementation of a cloud-based LLM for the defensive side (such as Claude), would entail additional cost, unless this was to be possessed anyway